Recommender systems: neural, and evaluated honestly

MovieLens 1M, four protocols, one model

LECTURE 22 LECTURE NOTES

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Last lecture predicted ratings

And it did it well: RMSE 0.8587 from a rank-16 factorisation, against 1.1185 for the global mean.

> **BUT NOBODY DEPLOYS A RATING PREDICTOR**
>
> A system does not fill in a matrix. It picks **ten items** out of a catalogue and puts them in front of a person. That is a *ranking* problem over items the user has *not* interacted with — and RMSE measures neither of those things.

So we change the question, keep the machinery, and discover that the change is not cosmetic.

Why an evaluation lecture

Every other lecture in this course spends most of its time on a method and a few minutes on how it was measured. This one inverts that, for a reason specific to the field:

- A large reproducibility study took eighteen recently published neural recommenders and found that most could be matched or beaten by well-tuned classical baselines
- The models were not fraudulent. The **comparisons** were broken — different protocols, untuned baselines, numbers quoted across papers
- And nothing in any individual paper looked wrong

✗ That is the failure mode this course exists to vaccinate against, arriving in its purest published form. It is worth thirty-five minutes.

Why this is the last lecture of Part V

Part V could have ended on an architecture. It ends on a measurement instead, and deliberately.

- The architectures converged in Lecture 20: a dot product of two learned vectors, in search and in recommendation alike
- What did not converge, and what still separates a usable system from a published one, is **knowing what your number means**
- And in both halves of this part, the ground truth turned out to be a record of what some earlier system chose to show

That is the thread from Lecture 1: a result you cannot state the conditions of is not a result. Part V is where it costs the most to forget.

Today

1. From rating prediction to top-N, and what the metric becomes (15 min)
2. **Two-tower models** — Lecture 20's bi-encoder with users in place of queries; negative sampling and the sampled-softmax correction (25 min)
3. **Evaluation** — the protocol as an experimental variable, measured (35 min)
4. Popularity bias, and what an offline number is worth (15 min)

No derivation. Part V's mathematics was in Lectures 19 and 21; this one is measurement, which is the other half of the course.

Where we are in Part V

Lecture 19	a query, a corpus, a ranking — and the metrics
Lecture 20	the same, with embeddings; and BM25 still won on NDCG
Lecture 21	no query; a user, a history, and a factorisation
Lecture 22	**the same problem, scored as a ranking — and scored honestly**

The architecture in this lecture is the one from Lecture 20. The metrics are the ones from Lecture 19. What is new is the **experimental design**, which is the part nobody teaches and everybody needs.

What you should be able to do afterwards

- Say why RMSE is the wrong metric for a recommender, in terms of what a recommender does
- Describe a two-tower model, and say what it shares with a bi-encoder
- Explain sampled softmax and the correction it needs
- State the difference between a random and a temporal split, and predict the direction of the difference
- Explain why sampled metrics are biased, and in which direction
- Read a recommender result and say what it does *not* establish

FIRST FIFTEEN MINUTES

From ratings to rankings

15 minutes

The task, restated

	Lecture 21	Lecture 22
Question	what would they rate it?	what should we show?
Output	a number per pair	an ordered list of ten
Evaluated on	ratings they gave	items they had not yet touched
Metric	RMSE	HR@10, NDCG@10 — Lecture 19's
Data	explicit ratings	implicit interactions

Row three is the one that changes everything. The evaluation set is no longer a sample of what the user did — it is a claim about what they *would* do, on items they never saw.

Precisely why RMSE fails here

Lecture 21's RMSE of 0.8587 was computed on *held-out ratings* — pairs where the user chose to watch the film and chose to rate it.

- A recommender operates on the complement of that set: films the user has **not** chosen
- RMSE weights every prediction equally, and a recommender only shows ten items — being accurate on the 3-star films costs nothing and buys nothing
- And it is a *pointwise* error, whereas showing ten items is a decision about *order*

> **A CONCRETE CASE**
>
> A model that predicts every rating 0.3 stars too low has a poor RMSE and **a perfect ranking**: a constant offset changes no order at all. The two metrics can disagree completely.

The data, binarised

To make MovieLens implicit, call a rating of 4 or 5 an interaction and discard the rest:

Ratings	1,000,209
Positive interactions	575,281
As a fraction of ratings	57.5%
Cells with no interaction	97.4%

✗ Note what the threshold did: a film a user rated 3 is now indistinguishable from one they never saw. We have thrown away the only genuine negatives in the dataset in order to simulate the situation of a real system, which never had them.

The metrics, from Lecture 19

- **HR@10** — hit rate: is the held-out item in the top ten? For one held-out item this is recall@10, and it is 0 or 1 per user
- **NDCG@10** — the same, discounted by position: $1/\log_2(\text{rank}+1)$ when hit, 0 otherwise

Both come straight from Derivation 19 with $|R| = 1$. Nothing new is needed, which is the point of having done retrieval first.

> **ONE RULE, CARRIED OVER AND ABOUT TO MATTER**
>
> An item the user has **already interacted with** is removed from the candidates before ranking. Recommending something already watched is not a hit, and leaving them in inflates every number on a popularity baseline especially.

HR@10, defined precisely

Precision matters here, because the whole lecture turns on small differences in definition:

1. Hold out exactly **one** interaction per user; call its item g
2. Score the candidate set for that user; remove every item they interacted with in the *training* data
3. Take the top ten. $\mathrm{HR} = 1$ if g is among them, else 0
4. Average over users — macro, one vote each

With one held-out item, HR@10 *is* recall@10, and NDCG@10 is $1/\log_2(\mathrm{rank}+1)$ when hit. Both come straight from Derivation 19 with $|R| = 1$.

✗ Step 2 is where the candidate set is defined, and the candidate set is half of what this lecture measures.

Why already-seen items must be excluded

A small implementation detail with a large effect, and a good example of how a protocol leaks.

- Leave them in and the popularity baseline recommends the blockbusters every heavy user has already watched — scoring nothing, but occupying the ten slots
- Leave them in for the factorisation and it recommends what the user already interacted with, because that is what it was trained to score highly
- Which of the two is hurt more depends on the model, so the choice is not neutral between them

We exclude them everywhere, and say so. It is the only line of the popularity baseline that varies by user, and without it the baseline would not be a baseline.

The baseline that must be in every table

Recommend the ten most popular items. To everybody. Every time.

- No personalisation. No model. No parameters
- Minus the items this user has already interacted with, which is the only thing in it that varies by user

It sounds like a straw man. It is not: in the reproducibility literature it beats a substantial fraction of published neural recommenders, and by the end of this lecture you will see exactly how that happens.

> **RULE 2, AGAIN, AND IT BITES HARDEST HERE**
>
> A recommender that does not beat popularity has not been shown to personalise anything.

THE METHOD

Two towers

25 minutes

It is Lecture 20, with the query replaced

$$\text{score}(u, i) = E_{\text{user}}(u) \cdot E_{\text{item}}(i)$$

	Lecture 20	Here
Left tower	the query text	the user — id, history, context
Right tower	the document	the item — id, genre, text
Precomputed	document vectors	item vectors
At request time	encode the query	encode the user

Identical architecture, identical indexability argument, identical approximate-nearest-neighbour problem. **Retrieval and recommendation converge here**, and that convergence is why Part V is one part rather than two.

What the towers can hold

- **Matrix factorisation is the special case** where each tower is a lookup table: $E_{\text{user}}(u) = p_u$, $E_{\text{item}}(i) = q_i$, and there the resemblance ends
- A tower can instead be a network over *features*: the item's genre, its text, its age; the user's recent history, device, time of day
- Which is what buys the cold start: a brand-new item has no q_i to look up, but it does have a genre and a description

THE HONEST SUMMARY OF "NEURAL RECOMMENDERS"

The architecture is a dot product of two learned vectors, exactly as before. What is new is **what the vectors are computed from** — and on datasets with no features beyond ids, there is nothing for the network to compute from, which is a large part of why the reproducibility results came out as they did.

Training: the softmax over the catalogue

$$\mathcal{L} = -\log \frac{\exp(s_{ui})}{\sum_{j \in \text{catalogue}} \exp(s_{uj})}$$

The natural objective: the observed item should win against every other item. It is the cross-entropy of Lecture 17 with the catalogue as the classes.

✗ And it is unaffordable. The denominator runs over 3,706 films here, and over tens of millions in a real catalogue — per interaction, per step.

So sample the denominator. Which introduces a bias, and the bias has a correction, and the correction is the examinable part.

Sampled softmax, and its correction

Draw m negatives from a proposal distribution q and sum over those instead. Popular items are drawn more often, so they are pushed down more often — the sample is not the catalogue, and the gradient knows it.

$$s'_{uj} = s_{uj} - \log q(j)$$

Subtracting $\log q(j)$ from each sampled negative's score makes the sampled sum an unbiased estimator of the full one. Without it, the model over-corrects against popular items and systematically under-recommends them.

It is the same correction as in word2vec's sampled objectives, and the same idea as importance weighting: reweight by how likely you were to have drawn the thing you drew.

In-batch negatives, once more

Batch B	Encoder passes	Scores	Negatives per user
8	16	64	7
32	64	1,024	31
128	256	16,384	127
512	1,024	262,144	511
2,048	4,096	4,194,304	2,047

Exactly Lecture 20's table, and the same arithmetic: encoding grows linearly, scores grow quadratically.

✗ With one difference that matters. In-batch negatives are drawn from the *interaction stream*, so an item's sampling probability is proportional to its popularity — which is precisely the $q(j)$ the correction needs. Skip the correction here and popularity bias is built into the gradient.

What training a tower actually involves

- A batch of observed (user, item) pairs from the interaction log
- Both towers run; scores are all pairs in the batch — the diagonal is the positives
- Cross-entropy along each row, with the $\log q$ correction subtracted from the off-diagonal entries
- Item vectors re-indexed periodically, because they drift as the tower trains — and a stale index is silently wrong, exactly as in Lecture 20

> **WHERE THE WORK REALLY IS**
>
> Not in the architecture. In the sampling distribution, the correction, the freshness of the index, and the protocol you evaluate it under — which is the same list as Lecture 20's, and it is why these two lectures are a pair.

Cold start, and what the towers buy

	Lookup table (MF)	Feature tower
New item	✗ no vector at all	encode its genre and text
New user	✗ no vector at all	encode context and first actions
Tail item	estimated from very few interactions	shares parameters with similar items

17.9% of films here have fewer than twenty interactions, so row three is most of the catalogue.

This — not accuracy on the head — is the honest case for a neural recommender. And it is invisible to a metric dominated by popular items, which is one more reason the protocol decides what progress looks like.

THE HEART OF THE LECTURE

The protocol is an experimental variable

35 minutes · examinable

Two decisions nobody reports

To evaluate a recommender you must decide two things, and papers routinely state neither:

1. **Which interaction is held out?** One chosen at random from the user's history, or their *last* one in time
2. **Against what is it ranked?** The whole catalogue, or the held-out item against a sample of, say, 100 items the user did not interact with

Two binary choices, four protocols. We are going to run the **same model on the same data** under all four.

> **COMMIT NOW**
>
> Write down how much you think the four numbers can differ. A few percent? A factor of two? Keep the number; you will want it in ten minutes.

Stating a protocol, in full

A complete protocol names six things. Most papers name two:

1. How the data was **filtered** — here, ratings of 4 or 5
2. What was **held out** — one interaction per user
3. How it was **chosen** — at random, or the most recent
4. What it is **ranked against** — the catalogue, or a sample
5. What was **excluded** from the candidates — already-seen items
6. The **metric and cutoff** — HR@10, NDCG@10

Change any one and the number changes. Three of the six are varied in the next few slides, and each one moves it more than most published modelling contributions do.

Decision 1 · Which interaction is held out

	Random (leave one out)	Temporal (leave last)
Held out	one interaction, chosen uniformly	the user's most recent
Training set contains	**✗ the user's future**	only their past
Matches deployment	**✗ no**	**yes ✓**
Prevalence in the literature	very common	less common

Under a random split the model may have been trained on films the user watched *after* the one it is being asked to predict. No deployed system ever has that, and it is a leak of exactly the kind Lecture 2 was about — just harder to see, because no column was duplicated.

The random split, as a leak

Lecture 2 defined a leak: information in the training data that will not be available at prediction time. Check the definition against the random split.

- The model is asked to predict interaction t
- It was trained on interactions $t+1, t+2, \ldots$ by that same user
- A deployed system has none of them, because they have not happened

> **IT IS A LEAK, AND IT LOOKS LIKE A CONVENTION**
>
> No column was duplicated, no target was copied, nothing was fitted before the split. The leak is in the *shape* of the evaluation, which is why it survived so long as a default — and why the effect is measured rather than argued in the next few slides.

A random split over interactions

FAILURE CONDITION

Split a user's interactions at random and the model is trained on what they did afterwards and tested on what they did before. The score improves, the protocol is the reason, and no error is raised because every row is genuinely that user's.

Decision 2 · Against what is it ranked

	Full catalogue	Sampled, 100 negatives
Candidates ranked	3,706	101
Cost	score every item	score 101
Matches deployment	**yes — the system ranks the catalogue ✓**	**✗ no**
Why anyone does it	—	it was expensive, once

Sampling was a computational shortcut from a time when scoring a catalogue was expensive. Ranking one held-out item against 100 random others is a far easier task than ranking it against 3,706 — and the 100 are drawn uniformly, so they are mostly obscure films nobody would recommend.

The experiment

- One model: a rank-32 factorisation of the binarised interaction matrix
- One dataset: MovieLens 1M, 575,281 positive interactions
- One baseline: recommend the most popular items, minus what the user has already seen
- Four protocols: {random, temporal} × {full, sampled 100}

Everything is held fixed except the protocol. Whatever the numbers do, **the model did not change**.

The model, stated so it can be dismissed

- A rank-32 factorisation of the binary interaction matrix — the simplest two-tower model there is, with both towers lookup tables
- Fitted once per split, then scored under both candidate protocols
- Nothing tuned per protocol, because tuning per protocol is one of the ways the comparisons in the literature broke

WHY A DELIBERATELY PLAIN MODEL

The claim of this lecture is not that this model is good. It is that **the protocol moves the number more than the model does**, and that argument is cleanest when the model is one you could have written in Lecture 21.

Predict the four numbers

Before the table. You have the pieces:

- 3,706 films in the catalogue, one right answer
- Sampling replaces that catalogue with 101 candidates
- A random split lets the model see the user's later choices

Which of the two decisions do you expect to matter more? And — the question that turns out to be the interesting one — do you expect the *ordering* of the two methods to survive both?

Both effects are large; one of them is much larger. The reason to guess first is to notice, afterwards, that the guess was not obvious — because a protocol that produces a wrong answer does not feel wrong from the inside.

The table

HR@10	Full catalogue	Sampled 100
Temporal · factorisation	✗ **0.0926**	0.6792
Temporal · popularity	0.0404	0.4864
Random · factorisation	0.2132	**0.8102** ✓
Random · popularity	0.0879	0.6186

✗ The same model on the same data scores 0.0926 and 0.8102 — a factor of 8.8. Nothing about the model changed. Only the two sentences describing how it was scored.

The same table, in NDCG

NDCG@10	Full catalogue	Sampled 100
Temporal · factorisation	✗ **0.0464**	0.4160
Temporal · popularity	0.0196	0.2656
Random · factorisation	0.1204	**0.5630** ✓
Random · popularity	0.0462	0.3709

Same shape, same conclusion: a factor of 12.1 across the corners.

Which is worth checking, because “use a better metric” is the natural first response to the HR table — and it does not help. The problem is not the metric; it is what the metric was computed over.

Take that apart · the split

HR@10, full catalogue	Temporal	Random	Ratio
Factorisation	0.0926	0.2132	✗ ×2.30
Popularity	0.0404	0.0879	✗ ×2.18

Switching from a temporal to a random split **more than doubles** both numbers, and does it to the personalised model and the unpersonalised baseline alike.

That last part is the tell. If the gain were the model learning something, it would not lift a popularity list by the same factor. It is the *task* getting easier: predicting a randomly chosen item from a history you have already seen most of.

How large are modelling gains, for comparison?

To read the protocol effects, you need a sense of what a *real* improvement looks like in this field.

- A new architecture reporting a 5–10% relative gain on HR@10 is a publishable result
- Our two protocol decisions moved HR@10 by **130%** and **641%** relative, on an unchanged model

✗ The undocumented choices are an order of magnitude larger than the documented contributions. That is the whole reproducibility problem in one comparison, and it is why the study found what it found.

Take that apart · the sampling

HR@10, temporal split	Full	Sampled 100	Ratio
Factorisation	0.0926	0.6792	✗ ×7.3
Popularity	0.0404	0.4864	✗ ×12.0

Sampling is the larger effect by far. HR@10 against 100 random negatives asks "is the right item in the top ten of **101**?" — and ten of 101 is already a tenth of the candidate set.

A *random* ranker scores 0.0990 under this protocol. Against the full catalogue it scores about 0.0027. The floor moved by a factor of 37, and every method rides up with it.

Why sampling flatters unequally

The 100 negatives are drawn uniformly, and the catalogue is mostly tail: the top 1% of films hold 8.7% of interactions.

- So a uniform sample of 100 contains, in expectation, about 1 of the head films — the only items a user might plausibly have chosen instead
- A **popularity** model is competing against almost no popular items, so it wins easily
- A model whose skill is discriminating *among plausible* items has nothing to discriminate

✗ The distortion therefore depends on what kind of model you have, which is exactly the condition under which a metric can reverse an ordering. A bias that were uniform would at least preserve rankings.

The consequence that ends careers

System, as reported	HR@10
Popularity, random split, sampled 100	**0.6186**
Factorisation, temporal split, full catalogue	✗ **0.0926**

✗ A method with no personalisation whatsoever, evaluated generously, reports 6.7 times the score of a real factorisation evaluated honestly.

Both numbers are correct. Both are computed without error. Put them in a table together, as papers do when they quote a baseline from another paper, and the comparison is **meaningless** — and nothing in either number says so.

What is stable across protocols

Protocol	Factorisation ÷ popularity
Temporal, full catalogue	**×2.29 ✓**
Random, full catalogue	×2.42
Temporal, sampled 100	×1.40
Random, sampled 100	×1.31

The absolute numbers spanned a factor of 8.8. The *ratio* to the baseline spans far less — from ×1.31 to ×2.42.

HENCE THE PRACTICAL RULE

Report the baseline in the same table, under the same protocol. It does not repair the protocol, and it does make your number readable by someone whose protocol differs — which is everyone.

Is sampling at least unbiased?

A natural hope: sampling adds noise but preserves the *ordering* of systems, so comparisons still hold. It does not, and the reason is structural rather than statistical.

- The 100 negatives are drawn **uniformly** from the catalogue, and the catalogue is mostly the long tail
- So the sample almost never contains a *popular* item — the only real competitors for a top-ten slot
- A model that is good at not recommending obscure films therefore looks excellent, and a model good at choosing among plausible ones gains nothing

> **SO THE DISTORTION IS NOT UNIFORM ACROSS MODELS**
>
> Sampling flatters some systems more than others, which means it can and does **reverse** the ordering. That is the finding the recommender literature had to absorb, and it invalidated a lot of published comparisons.

What the numbers actually license

- Under the honest protocol — temporal, full catalogue — the factorisation scores 0.0926 and popularity 0.0404. **The model is worth 2.3 times the baseline**, and that is the real result
- Every other number in the table is larger and means less
- NDCG@10 tells the same story: 0.0464 against 0.0196

> **THE HONEST HEADLINE**
>
> “A rank-32 factorisation places the user’s next film in the top ten **9.3%** of the time, against **4.0%** for a popularity list, under a temporal split with the full catalogue ranked.” Every clause is load bearing.

The mean hides the users

HR@10 per user is 0 or 1. A mean of 0.0926 therefore means precisely this: the held-out film was in the top ten for **9.3% of users** and not for the rest.

- There is no partial credit and no distribution to inspect — the mean *is* the proportion
- Which makes it honest, and makes its confidence interval computable: it is a proportion over 6,040 users
- NDCG@10 does have a spread, and it is rarely reported

Worth saying because “report the variance” is good advice that means different things for different metrics. For HR the right companion is an interval; for NDCG it is a distribution.

Why the honest numbers look so small

9.3% sounds like failure. It is not: the task is to place *one specific film* in the top ten of 3,706, from a history that ends before it.

Random ranker	0.0027
Popularity	0.0404
Factorisation	**0.0926**

Against chance, the factorisation is **34 times** better. Small absolute numbers are what an honest protocol on a hard task looks like.

✗ And a field that finds small numbers embarrassing will drift toward protocols that produce large ones. That drift is not fraud, and it does the same damage.

So what should you actually do?

1. **Rank the full catalogue.** It is affordable now; the shortcut is a historical artefact
2. **Split in time**, per user, and say so
3. **Include popularity** and a tuned classical baseline in the same table
4. **Never quote a number from another paper** into your table without re-running it under your protocol
5. **Report coverage and the popularity profile** beside the accuracy metric

All five are cheap. None of them is standard. Every one of them can only make your headline number smaller, which is the whole difficulty.

LAST FIFTEEN MINUTES

Popularity bias, and what an offline number is worth

15 minutes

FAILURE CONDITION — A POPULARITY LIST THAT BEATS THE MODEL

Under a sampled protocol a list that ignores the user entirely reports 6.7 times a real model’s hit rate. The number is computed correctly. It measures how the negatives were sampled, not whether anything understood a user.

The feedback loop

A recommender is not an observer of its data. It **produces** it:

1. The system recommends popular items, because they are the safe bet
2. Users interact with what they are shown — they cannot interact with what they are not
3. Those interactions become the next training set
4. Which makes the popular items more popular still

> **NOTHING IN THE OBJECTIVE NOTICES**
>
> Every offline metric in this lecture is computed on interactions produced by whatever system was running at the time. A model that reproduces the old system's behaviour scores well, and a model that would have shown something better scores badly — because the evidence for that something was never collected.

It is the pooling problem again

Lecture 19	Lecture 22
unjudged documents counted as irrelevant	unseen items counted as negatives
the pool came from the systems of its day	the log came from the recommender of its day
a novel method is penalised for finding new documents	a novel model is penalised for recommending new items

Three lectures apart, the same structure: **the ground truth records what a previous system chose to show, and everything else is scored as wrong.**

Which is why Part V ends here rather than on an architecture. The architectures converged two lectures ago; the measurement problem did not.

The long tail, and who is served

- The top 1% of films hold 8.7% of interactions, and 17.9% of films have fewer than twenty
- Recommending from the head is *correct* often enough to score well — popularity alone reaches 0.0404 honestly measured
- And it is worth nothing to the user, who had already heard of those films

THE METRIC CANNOT SEE USEFULNESS

HR@10 gives full credit for a correct popular recommendation and the same credit for a correct obscure one. If what you value is discovery, you must **measure something else as well** — coverage, the popularity distribution of what you recommend, the fraction of the catalogue ever shown.

What to report, beside the metric

- **Catalogue coverage** — what fraction of items are ever in anyone's top ten. A model with a great HR@10 and 2% coverage is a popularity list with extra steps
- **Popularity profile** — the distribution of recommended items' interaction counts, against the catalogue's
- **Per-user spread**, not just the mean — a mean HR of 0.09 can be most users at zero and a few at one
- **The protocol**, in the sentence that states the number

None of these is hard. They are absent from papers because nothing forces them to be present, and because each of them can only make the headline look worse.

Five questions for a recommender paper

1. **Full catalogue or sampled?** If sampled, the number is not comparable to anything
2. **Random split or temporal?** A random split leaks the user's future
3. **Is popularity in the table**, under the same protocol?
4. **Were the baselines tuned** as carefully as the proposed model, or quoted from elsewhere?
5. **Is there an online result?** If not, the claim is about a log, not about users

Five questions, and in the reproducibility study most papers failed at least two.

And then: none of it is the real test

Every number in this lecture is **offline**: computed on a log, about a system that was never run.

- The only real measurement is an **online experiment** with users, and it measures something different — whether people engaged, returned, cancelled
- Offline and online results correlate weakly, and the literature is candid about that
- An offline metric is a *filter*: cheap enough to reject bad ideas before they reach users, not evidence that a good one works

> **THE HONEST POSITION**
>
> Use offline evaluation to decide what is worth testing. Use an online test to decide what is worth keeping. **Do not use either to make the claim the other one supports.**

What an online test would measure instead

Offline	Online
did the model rank the held-out item highly?	did people click, watch, return, cancel?
on a log the old system produced	on traffic the new system produces
a proxy, computable in seconds	the objective, over weeks
no risk	real users see the worse arm

The last row is why offline evaluation exists at all, and the second row is why it cannot settle anything: the offline test asks whether you would have reproduced the old system’s successes.

One more thing the metric cannot see

A recommender chooses what a person is exposed to, at scale, and the objective it is optimised for is engagement measured by interaction.

- Content that provokes engages, and engagement is the signal the model is fitted to
- The feedback loop of six slides ago then amplifies whatever the objective rewards, including things nobody chose to reward
- None of this appears in HR@10, NDCG@10, coverage, or any metric in this lecture

> **STATED, NOT RESOLVED**
>
> This course cannot settle what a recommender should optimise. It can insist that you know your objective is a **proxy**, that you can say what it omits, and that "the metric improved" is never by itself an argument that the system got better.

Sequential recommendation, briefly

Everything so far treated a user as a *set* of interactions. But order carries information: someone who has just watched the first two films of a trilogy is in a state, not merely in possession of a history.

- Model the sequence, and predict the next item — which is the language-modelling objective of Lecture 17, with items as tokens
- The architectures are the ones you already have: a recurrent network (Lecture 16), or self-attention over the history (Lecture 18)
- And a temporal split becomes **mandatory**, because a random one destroys the very ordering the model is about

Not examinable beyond that paragraph. It is here because it is where the field went, and because it is the clearest case of Part IV's machinery arriving unchanged.

The convergence, in one slide

	Search (L20)	Recommendation (L22)
Left tower	query	user
Right tower	document	item
Score	dot product	dot product
Negatives	in-batch, hard-mined	in-batch, sampled
Index	ANN over documents	ANN over items
Ground truth	pooled judgements	a log from the old system

Six rows, and only the last two differ in substance. The two halves of Part V are one method with two data-collection problems, and the data-collection problems are the hard part in both.

The notebook for this lecture

Open Lecture 22 in Colab

- **Runs on free CPU** in a few minutes. The four protocols are four loops over the same fitted model
- The four-cell table reproduced, so you compute the factor rather than take it from a slide
- The popularity baseline built first, because it is four lines and it is the row that makes the rest legible

> **WHAT TO CHANGE**
>
> Drop the sampled negatives from 100 to 20 and re-run. Predict the direction first, then check whether the *ordering* of the two methods survives.

Vocabulary

Two-tower	user encoder and item encoder, scored by a dot product
Sampled softmax	a softmax whose denominator is estimated from a sample
logQ correction	subtracting $\log q(j)$ to unbias that estimate
Leave-one-out	hold out one interaction per user, chosen at random
Temporal split	hold out the most recent interaction instead
Sampled metric	rank the held-out item against a sample of negatives
Popularity bias	systematic over-recommendation of already-popular items
Feedback loop	the system shaping the data it will next be trained on

Scope

Scope

- why RMSE is the wrong metric for top-N; the two-tower model and its identity with Lecture 20's bi-encoder; sampled softmax and the $\log q$ correction; the four protocols, the direction of each effect, and why sampling can reverse an ordering; the popularity baseline; popularity bias and the feedback loop; the offline/online distinction — EXAMINABLE
- specific architectures and libraries, serving, A/B testing machinery — NOT EXAMINABLE — ENGINEERING
- sequential and session-based models in detail, counterfactual and off-policy evaluation, bandits for exploration — BEYOND THE SYLLABUS

Summary

1. A recommender ranks a catalogue for a user, so it is scored by Lecture 19's metrics on items the user had not touched — not by RMSE
2. The two-tower model is Lecture 20's bi-encoder with users in place of queries, and matrix factorisation is its lookup-table special case
3. The catalogue softmax is unaffordable, so it is sampled — and a sampled softmax needs the $\log q$ correction or popularity bias enters the gradient
4. The **protocol** moved HR@10 from 0.0926 to 0.8102 on one unchanged model, and it lets a popularity list report 6.7 times a real model's honest score
5. Offline numbers filter ideas; only an online test measures the thing you care about

FAILURE CONDITION — THE PROTOCOL, NOT THE MODEL

One unchanged model reports HR@10 of 0.0926 or 0.8102 depending only on how the candidates were sampled. A recommender's headline is a property of the evaluation protocol at least as much as of the model, and the protocol is the half that goes unreported.

One line to keep

> THE LINE
>
> **The protocol is part of the result. A number without it is not a measurement.**

One model, one dataset, four sentences describing how it was scored, and a factor of 8.8 between the extremes. Every one of those four numbers is arithmetically correct, and three of them would mislead anyone who read only the headline.

Three numbers to leave with

Popularity, honestly measured	0.0404
A rank-32 factorisation, honestly measured	**0.0926**
Popularity, measured generously	✗ **0.6186**

Rows one and two are the finding: a real model is worth **2.3 times** the trivial baseline. Row three is the same trivial baseline, scored under a protocol many papers use, reporting 6.7 times the honest model.

✗ If you remember one thing from Part V, make it that row three exists and looks exactly like a result.

Where students lose marks on this material

- Comparing HR@10 figures from two papers without checking the protocol. This lecture exists to make that reflex automatic
- Saying sampled metrics are “noisier but unbiased”. They are biased, and non-uniformly across models
- Omitting the popularity baseline
- Forgetting to remove already-seen items before ranking
- Describing a two-tower model as fundamentally different from matrix factorisation. It is the same score with richer encoders

An exam-style question, worked

“A paper reports HR@10 of 0.8102 for its new model, against 0.0404 for a popularity baseline quoted from an earlier paper. What, if anything, has been established?”

- **Nothing.** The two numbers are from different protocols, and a quoted baseline is by definition not run under yours
- Under one protocol, popularity itself reaches 0.6186 — larger than the number being celebrated’s honest counterpart
- The repair costs nothing: re-run popularity under the paper’s own protocol and put it in the same table

Section B of the paper is this shape: a table, a claim, and the question of what the table supports.

Part V, closed

Lecture 19	a ranking is scored by a question, chosen before the scores
Lecture 20	the first stage sets the ceiling; the old method still won
Lecture 21	the missing entries are the data; do not impute them
Lecture 22	**the protocol is part of the result**

Four lectures outside the textbook, and not one of them needed a method you had not already met. What they needed was the discipline of Part I applied to problems whose ground truth is itself a measurement.

What Part VI does with all this

- Lecture 23 takes the contrastive objective of Lecture 20 and puts an **image** on one side of it — and uses Lecture 19's metrics to score the result
- Lecture 24 puts the retriever of Lecture 20 in front of a **generator**, where a first stage that misses a document becomes a fluent, confident, wrong answer
- Both are back inside the textbook. Both depend on Part V being understood, which is why it sits where it does

And in both, the question that decides everything is the one this lecture spent thirty-five minutes on: *measured how?*

The Part V checklist

1. Name the metric and the cutoff **before** seeing scores
2. Fit the trivial baseline first — corpus order, popularity, the global mean
3. State the protocol in the same sentence as the number
4. Ask where the ground truth came from, and what it counts as negative
5. Report what the metric cannot see, in one line

Five habits, four lectures, two datasets. They are not specific to search or recommendation — Part V is simply where their absence is most visible and most expensive.

Reading

- **These lecture notes are the primary source** — lecture-22.pdf, the extended notes for this lecture. The textbook does not cover this material
- Dacrema, Cremonesi and Jannach, *Are We Really Making Much Progress?* — the reproducibility study; read it before you believe any recommender leaderboard
- Krichene and Rendle, *On Sampled Metrics for Item Recommendation* — why sampled metrics are not merely noisy
- Yi et al., *Sampling-Bias-Corrected Neural Modeling* — the $\log q$ correction in a deployed two-tower system

Examinable content is what is on these slides and in the notebook.

Before the next lecture

- Run the notebook and reproduce the four-cell table. It is the single most useful thing in Part V for reading other people’s work
- Part VI returns to the textbook, and to images — but the contrastive objective of Lecture 20 comes with it, now with a picture on one side and a caption on the other
- And Lecture 24 puts the retriever of Lecture 20 in front of a generator, where every failure of Part V reappears with a fluent voice

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 22

five, in the style of the written paper

Exercises — Lecture 22

1 A recommender is scored by RMSE on held-out ratings. State why that is the wrong metric, using the set the model actually operates on. [5]

2 The same model scores HR@10 of 0.0926 and 0.8102 on the same data. Name the two decisions that differ, and which moves the number more. [5]

3 Explain why a sampled metric is not merely noisier than a full-catalogue one, and give the consequence for comparing two published systems. [5]

Solutions on Lecture 23’s deck.

Exercises — Lecture 22 (continued)

4 A sampled softmax draws negatives in proportion to popularity. State the correction, and what enters the model if it is skipped. [4]

5 A recommender is trained on interactions produced by an earlier recommender. Name the effect, and the earlier lecture whose problem it repeats. [4]

Solutions on Lecture 23’s deck.

THE FIVE SET AT THE END OF LECTURE 21

Solutions Lecture 21

not part of this lecture

Solutions — Lecture 21

1. State why the truncated SVD cannot be applied directly to a ratings matrix, naming the condition of…

✓ The Frobenius norm sums over every entry, and 95.5% of them have no value.

- The theorem is about one fully specified matrix
- Filling the gaps changes the problem into a different one with a different answer

2. A rank-32 SVD of the zero-filled matrix scores worse than predicting one number for everybody. Explain how…

✓ It is optimal for the matrix it was given, and that matrix is mostly zeros.

- Ratings run from 1 to 5, so a zero is off the scale rather than low
- The approximation spends its capacity reproducing an assumption nobody made deliberately

Solutions — Lecture 21 (2)

3. Write the ALS half-step for one user, and identify it as a familiar estimator.

✓ $p_u = (Q_u^\mathsf{T} Q_u + \lambda I)^{-1} Q_u^\mathsf{T} r_u$ — a ridge regression.

- Q_u contains only the items that user rated, so the missing entries never enter the system
- The system is $d \times d$ however many items the user rated

4. For any invertible M, $(PM)(QM^{-\mathsf{T}})^\mathsf{T} = PQ^\mathsf{T}$. State what this settles about…

✓ The predictions are determined and the factors are not, so an individual dimension has no meaning.

- The objective cannot distinguish P from PM
- A factor plot claims something the model is not claiming

Solutions — Lecture 21 (3)

5. Implicit feedback has no negatives. State why an objective over observed entries alone fails, and name the…

✓ It is satisfied by predicting yes everywhere. Weighted least squares over all cells, or a pairwise ranking objective.

- Only positives exist, so there is nothing to push down
- The first keeps a closed form and needs a confidence function; the second models only differences

LECTURE 23 · GÉRON, CHAPTERS 15–16

Vision transformers and multimodal retrieval

An image as a sequence of patches, and what that gives up
the contrastive objective and its temperature
Text and images in one space — and Lecture 19's metrics, used again

Part VI begins